VisionPriority AI — Review 2: Detection & Tracking handoff

- **From:** Harini (23BCE2170), Detection & Tracking pipeline
- **To:** Parvati (risk logic, integration, dashboard) · cc Aishwarya (dataset pipeline)
- **Status:** Stages 1–4 (Input → Preprocess → Detect → Track) are implemented, tested and validated. Parvati can start building the risk layer on this output today.

---

1. What is done (against the Review 2 task list)

Task (from our breakdown)	Status	Where
YOLO11s, Ultralytics, COCO-pretrained, 640 input	Done	`vp_perception/detector.py`, `config.py`
Consume 4 synchronized views, run once per view, every frame	Done. All 4 views go through **one batched forward pass** per frame	`frame_source.py`, `pipeline.py`
Output class, bbox, confidence per detection	Done	`contract.py`
ByteTrack **per camera view** (no cross-camera)	Done. 4 independent trackers	`tracker.py`
Check that two-stage association + low-confidence recovery helps under occlusion	Done on MOT17 ground truth, **with an honest result** (§6.2)	`scripts/eval_mot17.py`
Add track_id, centroid, bbox area, track age to the output	Done, plus area/centroid **history** for A and D	`tracker.py`, `contract.py`
Re-run YOLO11 n/s/m on our own data instead of quoting Ultralytics	Done on MOT17 (the only dataset we have with box labels)	§6.1
Spot-check ByteTrack ID consistency against MOT17 GT	Done: metrics + an annotated video	§6.2, `results/mot17/MOT17-09_bytetrack.mp4`
Test the Harini → Parvati handoff on real tracked frames	Done: 0 contract errors on every scene; A/D/P/Z raw signals computed from real output	§4, `examples/handoff_signals.py`

---

2. Run it (5 minutes)

```
pip install torch torchvision              # GPU build from pytorch.org if you have an NVIDIA GPU
pip install -r requirements.txt
python tests/test_perception.py            # expect 5 x "ok"
python scripts/run_nuscenes.py --videos <folder with sceneXX_CAM_*.mp4> --scenes scene00 --out results/nuscenes
python scripts/check_contract.py results/nuscenes/scene00/tracks.jsonl     # expect PASS
```

Already-computed outputs for scenes 00, 01, 02 and 04 are in `results/nuscenes/`, so **you don't need to run the detector to start building the risk layer**. Read `tracks.jsonl` directly.

3. The Output Contract (field by field)

Every tracked object, in every view, in every frame, produces one record (a plain `dict`). The first eight fields are **exactly** the contract from Slide 6, with exactly these names:

Field	Type / units	Meaning	Used for
`view`	str	nuScenes channel: `CAM_FRONT`, `CAM_BACK`, `CAM_FRONT_LEFT`, `CAM_FRONT_RIGHT`	Z, alert location text
`track_id`	int	ByteTrack ID, unique **only within its view**. Not consecutive (see §5)	identity
`class`	str	COCO name: person, bicycle, motorcycle, car, bus, train, truck, cat, dog, horse, sheep, cow, elephant, bear	C
`bbox`	[x1, y1, x2, y2] float, px	Box in the **original 1600×900 image**, clipped to the image	Z
`confidence`	float 0–1	Detector score of the box matched **this frame**	optional weighting
`centroid`	[cx, cy] float, px	Box centre	Z, D
`bbox_area`	float, px²	(x2−x1)·(y2−y1)	A
`track_age`	int, frames	frames since this ID first appeared; 1 on its first frame	P

Extra fields, so you don't have to recompute anything:

Field	Type / units	Meaning
`frame_idx`	int	synchronized-batch index, the same for all 4 views of one instant
`timestamp`	float, s	seconds since scene start
`view_name`	str	front / rear / front_left / front_right (for alert text)
`track_label`	str	display ID, e.g. `FL-12` (F, B, FL, FR + id), matching the Slide 9 style
`class_id`	int	COCO id
`class_group`	str	`vulnerable_road_user` (person, bicycle, motorcycle) · `vehicle` · `animal`
`image_size`	[w, h]	frame size: divide by this to get resolution-independent coordinates
`bbox_area_norm`	float	bbox_area / (w·h)
`track_age_s`	float, s	track_age / 12 fps
`hits`	int	frames in which the ID was actually matched (≤ track_age; lower = it was lost and re-found)
`first_frame_idx`	int	frame where the ID started
`centroid_history`	[[t, cx, cy], ...]	last 30 samples (2.5 s), oldest → newest, **includes the current frame**
`area_history`	[[t, area], ...]	last 30 samples, oldest → newest, **includes the current frame**

File format: `results/nuscenes/<scene>/tracks.jsonl`

One JSON object per line, one line per synchronized frame:

```
{"frame_idx": 80, "timestamp": 6.6667, "sync_mode": "proportional",
 "source_index": {"CAM_FRONT": 80, "CAM_BACK": 77, "CAM_FRONT_LEFT": 80, "CAM_FRONT_RIGHT":
78},
 "timing_ms": {"detect": 701.0, "track": 5.1, "total": 706.1, "n_detections": 41},
 "tracks": [
   {"view": "CAM_FRONT_LEFT", "track_id": 60, "class": "person",
    "bbox": [...], "confidence": 0.83, "centroid": [...], "bbox_area": ..., "track_age": 13,
    "frame_idx": 80, "timestamp": 6.6667, "view_name": "front_left", "track_label": "FL-60",
    "class_id": 0, "class_group": "vulnerable_road_user", "image_size": [1600, 900],
    "bbox_area_norm": 0.0069, "track_age_s": 1.083, "hits": 12, "first_frame_idx": 68,
    "centroid_history": [[5.6667, ..., ...], ...], "area_history": [[5.6667, ...], ...]}
 ]}
```


`tracks_flat.csv` has the same records, one per row (contract fields + key extras), for Excel or pandas. `results/handoff_sample_scene00.json` is a small 13-frame slice you can use for unit tests.

---

4. How each risk factor maps onto these fields

The formula, weights and thresholds are Parvati's to implement (Risk = 0.25C + 0.20Z + 0.30A + 0.15D + 0.10P; LOW < 0.35 ≤ MEDIUM < 0.65 ≤ HIGH). Below is where each input comes from, and a raw signal that is **already shown to work on real tracker output** (`examples/handoff_signals.py`).

Factor	Source fields	Raw signal that works (you still normalize it to 0–1)
C class vulnerability	`class`, `class_group`	lookup table, e.g. person/bicycle/motorcycle high, animal medium, vehicle by size
Z zone criticality	`view`, `centroid`, `bbox`, `image_size`	zones per view in normalized coords (cx/W, y2/H). The box **bottom edge y2/H** is the best "how near" proxy (closer objects sit lower in the frame).
A approach rate	`area_history`	slope of **log(area)** vs time over the last ~1 s. >0 growing (approaching), ≈0 static, <0 receding. +0.69/s = area doubles every second. `area_growth_per_s()` in the example.
D direction alignment	`centroid_history`, `view`	centroid velocity (px/s) over the last ~1 s, `centroid_velocity()` in the example. "Toward the vehicle" depends on the view: e.g. in FRONT_LEFT, moving right/down = toward the vehicle's path.
P persistence	`track_age`, `hits`	e.g. min(track_age_s / 1.0, 1). `hits/track_age` tells you if it was lost and re-found.

Real values from scene00, frame 80 (from `examples/handoff_signals.py`):

```
label    class        conf  age hits  cx/W  cy/H  area%  growth/s  vx px/s  vy px/s
FL-60    person       0.83   13   12  0.44  0.49   0.69    0.944   -198.0    -23.3   <-
growing fast: approaching
FR-81    car          0.45   39   38  0.81  0.59   0.36    0.600    286.9     37.7
B-47     truck        0.62   45   45  0.66  0.55   0.70   -0.002     18.5    -18.8   <-
steady, following behind
FR-54    car          0.88   48   48  0.41  0.58   1.22   -0.481    328.8     13.8   <- moving
away
F-344    person       0.85    1    1  0.87  0.62   1.08    0.000      0.0      0.0   <- brand-
new track: A=D=0 until ~3 frames
```

---

5. Behaviour you must know before building on it

1. **IDs are per view.** `CAM_FRONT` ID 12 and `CAM_FRONT_LEFT` ID 12 are different objects; the same car seen by two cameras has two IDs (cross-camera re-ID is out of scope, Slide 6). Use `(view, track_id)` or `track_label` as the key.
2. **IDs are not consecutive.** ByteTrack uses up an ID for every tentative track, including ones that are never confirmed, so you'll see `F-344` in a scene with 86 front tracks. That's normal.
3. **Only currently matched tracks are emitted.** A track that's temporarily lost isn't in that frame's list. If it comes back within 30 frames (2.5 s) it keeps its ID, and `track_age` keeps counting (so `hits` < `track_age`). For the dashboard, keep the last alert for a lost ID for a few frames instead of letting it flicker off.
4. **New tracks have no motion yet.** With fewer than 3 history samples, A and D can't be computed, so treat them as 0. This is also what P (persistence) is for: about 20% of IDs in a busy scene last only one frame (mostly distant pedestrians), and P should stop those from becoming HIGH alerts.
5. **`confidence` can be as low as 0.10.** These are boxes ByteTrack recovered in its second stage (the low-confidence matches). They belong to an existing track, so keep them; don't filter on confidence ≥ 0.25 or you lose exactly the continuity we picked ByteTrack for.
6. **`class` is a majority vote over the track's life**, so it doesn't flicker car ↔ truck frame to frame. The current-frame class isn't exposed; set `TrackerConfig.class_vote=False` if you ever need it.
7. **Coordinates are in original 1600×900 pixels**, not 640×640. YOLO letterboxes internally (aspect ratio kept, no stretching) and maps boxes back. So the OpenCV "resize to 640×640" preprocessing stage is handled inside Ultralytics; don't resize before calling `process()`.
8. **Frame sync is approximate for now.** See §7. `sync_mode` in every line tells you which mode was used.

Calling it live (integration)

```
from vp_perception import MultiViewVideoSource, find_scene_videos
from vp_perception.pipeline import PerceptionPipeline

pipe = PerceptionPipeline()   # PerceptionConfig(detector=DetectorConfig(device="0")) to force
GPU
for batch in MultiViewVideoSource(find_scene_videos("nuscenes_videos", "scene00")):
    result = pipe.process(batch)          # result.tracks -> list of contract dicts
    scored = risk_layer(result.tracks)    # Parvati: C, Z, A, D, P -> score -> priority ->
explanation
    dashboard.update(batch.frames, scored)
```

`result.timing_ms` gives detect / track / total milliseconds per frame. Add your own stage times to it for the ≤ 100 ms latency measurement.

---

6. Validation results

All numbers below were measured by running the code in this folder, not copied from papers.

6.1 Detector: YOLO11 n / s / m on our own frames

MOT17 is the only dataset we have with box labels (nuScenes videos came without 2D annotations), so detector accuracy is measured as **person AP on MOT17**: every 3rd frame of all 7 sequences, standard MOTChallenge ignore regions, 640 input. Latency was measured on the cloud test machine: **2-core CPU, no GPU**.

Model	Person AP50	Person AP50-95	Recall @conf 0.25	CPU ms / image (640)	Deck's cited T4 GPU ms	Params
YOLO11n	0.492	0.264	0.415	58	1.5	2.6M
YOLO11s	**0.537**	0.300	0.461	154	2.5	9.4M
YOLO11m	0.542	0.310	0.446	402	4.7	20.1M

Reading it: going n → s raises person AP50 from 0.492 to 0.537 (+4.5 pts) for 2.6× the time; going s → m adds only +0.5 pts for 2.6× the time. The ordering matches the published COCO numbers, so the s-variant choice holds on our data: most of the accuracy of m at roughly 38% of its cost. Run 4 views per frame and m is 1608 ms per frame on this CPU.

6.2 Tracker: ByteTrack on MOT17 ground truth (all 7 sequences, every frame, YOLO11s detections)

Compared against the same tracker with the low-confidence second stage turned off (boxes < 0.25 discarded), which isolates the exact feature the deck gives as the reason for choosing ByteTrack.

Metric	ByteTrack two-stage (ours)	Low-score stage off	Change
MOTA ↑	0.369	0.351	+1.8 pts
IDF1 ↑	0.453	0.438	+1.5 pts
Recall ↑	0.431	0.403	+2.8 pts
ID switches ↓	546	594	-8%
Fragmentations ↓	886	1324	-33%
Mostly tracked ↑ (of 546 GT people)	111	108	+3%
Mostly lost ↓	222	235	-6%
False positives ↓	6451	5285	+22%
Recall on partially occluded people (vis < 0.5) ↑	0.109	0.097	+1.2 pts
Same ID after full occlusion (vis < 0.3, ≥ 3 frames) ↑	0.372 (76/204)	0.400 (82/205)	-2.8 pts

Per sequence (two-stage): MOT17-02 MOTA 0.25 / IDF1 0.34, MOT17-04 MOTA 0.37 / IDF1 0.48, MOT17-05 MOTA 0.45 / IDF1 0.57, MOT17-09 MOTA 0.57 / IDF1 0.52, MOT17-10 MOTA 0.40 / IDF1 0.39, MOT17-11 MOTA 0.51 / IDF1 0.58, MOT17-13 MOTA 0.28 / IDF1 0.39

What this shows:

- The two-stage association does what the deck claims for **partial occlusion and weak detections**: fragmentations drop 33% (1324 → 886), ID switches drop 8% (594 → 546), recall rises 2.8 pts, MOTA rises 1.8 pts and IDF1 1.5 pts. The cost is 22% more false-positive boxes, which the risk layer's persistence factor (P) is there to damp.
- It does **not** bring IDs back after a **full** occlusion. When a person is hidden (visibility < 0.3 for 3+ frames) and reappears, the same ID returns in 37% of cases with the two-stage tracker vs 40% without it, so no real difference. Recovering identity after full occlusion needs appearance features (re-ID, e.g. BoT-SORT/DeepSORT), which ByteTrack doesn't use. For Review 2 we should say this honestly: "ByteTrack reduces fragmentation under partial occlusion; full-occlusion re-identification is future work."
- Absolute MOTA is low compared with ByteTrack's paper (≈80) because that number uses a detector **trained on MOT17** at high resolution. We use a COCO-pretrained YOLO11s at 640 on 1920×1080

crowds where many people are tiny or heavily occluded (person AP50 ≈ 0.54 above). Detection, not association, is the bottleneck.
- Visual spot-check: `results/mot17/MOT17-09_bytetrack.mp4`. Each ID keeps one colour, so an ID switch shows up as a colour change.

6.3 nuScenes: 4-view run (YOLO11s + 4 × ByteTrack)

Scene	Frames	Frames per view (F / B / FL / FR)	Unique track IDs	Avg active tracks / frame	IDs lasting ≥ 1 s	Main classes	CPU detect, 4 views (median)
scene00	224	224 / 216 / 224 / 217	279	14.17	78	person 146, car 91, truck 30, motorcycle 5	640 ms
scene01	229	229 / 229 / 231 / 233	377	28.65	150	car 204, person 137, bicycle 27, truck 7	602 ms
scene02	237	237 / 235 / 239 / 239	143	21.81	81	car 85, person 33, truck 14, bus 9	644 ms
scene04	237	237 / 235 / 236 / 236	107	10.92	51	car 74, person 14, truck 13, bus 5	650 ms

Videos: `results/nuscenes/<scene>/surround_tracks.mp4` (front-left · front · front-right on top, rear below).

Contract check: **0 errors** across all records in all scenes (`scripts/check_contract.py`).

6.4 Latency vs the PDS-32 target (≤ 100 ms)

On the 2-core CPU test machine, one 4-view frame takes ≈ 634 ms to detect and ≈ 5 ms to track. That **does not** meet 100 ms on CPU. ByteTrack is negligible; the detector is the whole cost. Ultralytics' published T4 TensorRT figure for YOLO11s is 2.5 ms per image, which would put 4 views at ~10–20 ms on a GPU. **Before Review 2, run `scripts/benchmark_latency.py` on the demo laptop** and put that real number on the slide. If the demo machine has no GPU, the fallbacks are YOLO11n (see 6.1), `imgsz=480`, or running the detector every 2nd frame and letting ByteTrack's Kalman prediction fill the gaps.

---

7. What we need from Aishwarya (dataset side)

1. **Timestamps for exact sync (most important).** The 4 videos of a scene have different frame counts (scene00: front 224, rear 216, front-left 224, front-right 217) and no timestamps, so frame *N* of each video is not the same instant. Right now the code aligns them proportionally (worst case ≈ 0.3 s off). Please export, next to each video, a `sceneXX_CAM_YY_timestamps.csv` with the columns `frame_idx,timestamp_us` (the nuScenes `sample_data.timestamp` of each frame you stitched). As soon as those files are in the folder, the loader switches to exact timestamp sync automatically. No code change needed.
2. `scene0_CAM_FRONT.mp4` is a duplicate of `scene00_CAM_FRONT.mp4` (same size). Delete it to avoid confusion.
3. The deck says nuScenes-mini for development and trainval for evaluation. The 10 scenes we received look like the mini split. If trainval scenes are added, keep the same `sceneXX_CAM_YY.mp4` naming and they'll work unchanged.
4. If her data loader yields frames directly (e.g. from nuscenes-devkit images instead of videos), it only has to produce `SyncedFrameBatch(frame_idx, timestamp, frames={view: BGR image})`. That's the whole interface (`vp_perception/frame_source.py`).

8. Settings used (all in `vp_perception/config.py`)

Setting	Value	Why
Model	`yolo11s.pt` (COCO)	deck choice; see 6.1
Input size	640 (letterboxed)	deck choice
Detector conf	0.10	ByteTrack's second stage needs low-score boxes
NMS IoU	0.70	Ultralytics default
Classes kept	person, bicycle, car, motorcycle, bus, train, truck, 7 animals	PDS-32: vehicles, humans, animals
ByteTrack high / low / new	0.25 / 0.10 / 0.25	Ultralytics defaults
Track buffer	30 frames (2.5 s at 12 fps)	how long a lost ID is kept
Match threshold	0.8	default
History length	30 samples	enough for 1–2 s motion windows